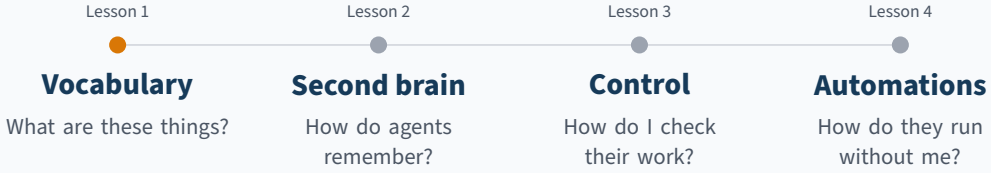
Agentic AI for Economic Research

Lesson 1: The vocabulary

Marco Le Moglie, Catholic University of the Sacred Heart
Four lessons of two hours, autumn 2026
github.com/MarcoLeMoglie/demo-lesson1

The course in four lessons



Today is the map. The next three lessons build on these words.

Twelve words for today

What an agent is	What it sees	How you extend it
model	prompt	rules file
harness	context window	MCP
agent	token	skill
tool and permission	compaction	sub-agent

By the end you should be able to define each one in a sentence. The last slide checks.

From chat to agent

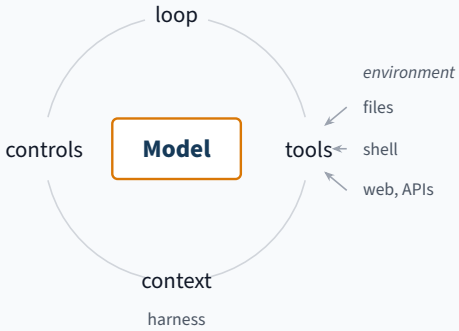
What changes when the model can act

Model plus harness equals agent

Harness

The runtime layer that wraps a language model and turns it into an agent that can act: it runs the loop, exposes tools, manages context and enforces controls.

Macedo (2026); Barbaste et al. (2026)



Not a harness: an agent framework such as LangChain, or an evaluation harness such as SWE-bench's. Across eleven production harnesses, none imports a general agent framework.

Same model, different harness: a different agent.

Chatbot vs agent

	Chatbot	Agent
Who runs the loop	You	The harness
Unit of delegation	An answer	A task
Files	Pasted or attached	Read, edited, created
Failure mode	A wrong *answer*	A wrong *action*
Your role	Reader	Supervisor

Different failure mode, different control.

The loop, in a real trace

Observe

reads the do-file

Act

runs Stata in batch

Observe

exit code 0, yet the log says `prise not found`

Act

edits one line

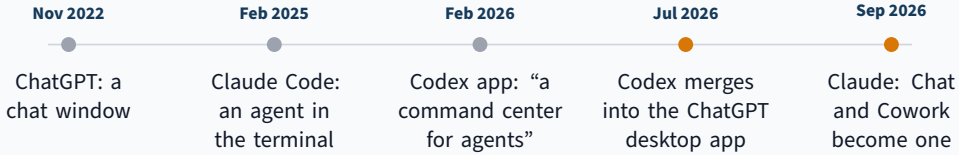
Verify

reruns, shows the end of the log, as `AGENTS.md` asks

```
> build · deepseek-v4.1-flash-free
→ Read code/analysis.do
$ command -v stata-mp || ls .../MacOS/stata-mp
.../MacOS/stata-mp
$ .../stata-mp -b do code/analysis.do; echo "EXIT:$?"
EXIT:0
→ Read analysis.log
← Edit code/analysis.do
-gen lprice = ln(prise)
+gen lprice = ln(price)
$ .../stata-mp -b do code/analysis.do; echo "EXIT:$?";
  tail -20 analysis.log
EXIT:0
[...]
(output written to output/table1.tex)
Fixed typo in `code/analysis.do:11`
(`ln(prise)` → `ln(price)`), reran the do-file,
and confirmed `output/table1.tex` exists.
```

Real run on the course demo repository, 26 Oct 2026: OpenCode with DeepSeek V4.1 Flash (free tier), 22 seconds. Paths shortened.

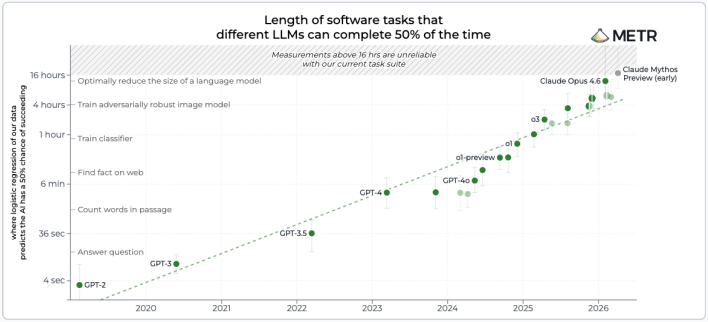
From chat window to command center



The interface no longer tells you whether you are chatting or delegating.

Source: OpenAI (2026a); Anthropic (2026); OpenAI, *Introducing the Codex app*, 2 Feb 2026.

How long a task can an agent finish?



METR, Time Horizon 1.1. Captured 26 Oct 2026.

What it measures

The length of software tasks, timed on human experts, that a model completes half of the time.

What it does not

How long an agent works alone. Economics tasks. Anything above 16 hours, which METR calls unreliable.

Status

Page frozen on 8 Sep 2026.

Live: chat vs agent

What we do

The do-file in the demo repository fails.

First, paste it into a chat window and ask for the bug.

Then give the same sentence to an agent opened in the repository folder.

What to look at

Who runs Stata.

Who reads the log.

What you have to check at the end.

Live on screen share. Demo repository: `https://github.com/MarcoLeMoglie/demo-lesson1`. If the demo fails, the next slide shows the recorded result.

Recorded result: what the agent left behind

The change

```
- gen lprice = ln(prise)
+ gen lprice = ln(price)
```

One line in the do-file. The data are untouched, as the rules file asks.

The table runs, but look at the empty base-category rows: “done” still needs a reader. Lesson 3.

The output

	(1) lprice	(2) lprice
l100km	0.0217	0.0193
	(0.0309)	(0.0331)
Weight (lbs.)	0.000377*	0.000406*
	(0.000145)	(0.000153)
Domestic	0	0
	(.)	(.)
Foreign	0.503***	0.486***
	(0.102)	(0.140)
Repair record 1978=1		0
		(.)
Repair record 1978=2		0.0372
		(0.132)
Repair record 1978=3		0.0793
		(0.0923)
Repair record 1978=4		0.102
		(0.114)
Repair record 1978=5		0.146
		(0.141)
Constant	7.095***	6.942***
	(0.199)	(0.215)
Observations	74	69

Standard errors in parentheses

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

`output/table1.tex` as written by the agent, compiled unedited.

Core vocabulary

What the agent sees, and how you extend it

Prompt vs task

Prompt

"Explain how to cluster standard errors in Stata."

One question, one answer. No state, no side effects.

Task

"Rerun both models with clustered errors, regenerate the table, tell me what changed."

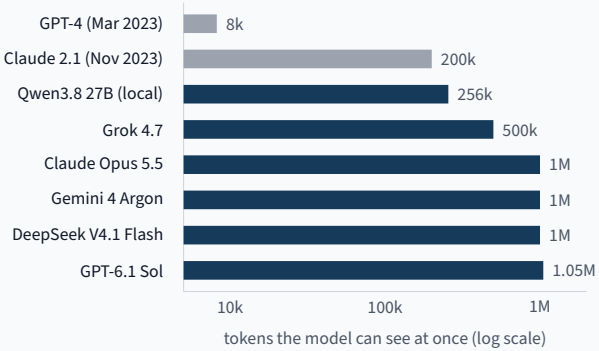
A goal, a workflow, an artefact you can check.

A task an agent can finish names four things

Goal	what should exist at the end
Inputs	which files, which data
Limits	what must not change
Done	how you will check it

Agents change the unit of delegation.

Context window and tokens



Token

The unit models read, write and bill. A 30-page paper is roughly 25k.

Context window

Everything the model sees in one turn: rules, files, tool output, history.

Context rot

Accuracy falls as the window fills. A bigger window is not a better memory.

Source: vendor model pages and Artificial Analysis, read 26 Oct 2026; `context_windows.csv`.

Compaction

Compaction

Taking a conversation nearing the context window limit, summarizing its contents, and reinitiating a new context window with the summary.

Anthropic (2025). In Codex and Claude Code: `/compact`. Most harnesses also do it on their own.



A summary loses detail. Decisions belong in files.

Rules files: prompts that stay

```
# Instructions for coding agents
[...]
## Layout

- `data/auto.dta`: raw data. Never modify or overwrite.
- `code/analysis.do`: the only analysis script.
- `output/`: generated tables. Safe to overwrite.
[...]
The log is written to `analysis.log` in the repository root.
Read it after every run: Stata reports errors there, not on
the terminal, and the exit code is 0 even when the do-file fails.

## Rules

1. Change the do-file, never the data.
2. Keep variable names short and lower case.
3. Report robust standard errors (`vce(robust)`) unless asked.
4. Before saying a task is done, run the do-file and show the
   last 20 lines of the log.
5. Do not install Stata packages without asking.
```

`AGENTS.md` of the course demo repository (excerpt). The agent reads it at every session start.

Where the agent is

Layout of the project, what is raw and what is output.

What it must not do

Never touch the data. No new packages without asking.

What "done" means

Run the do-file and show the end of the log.

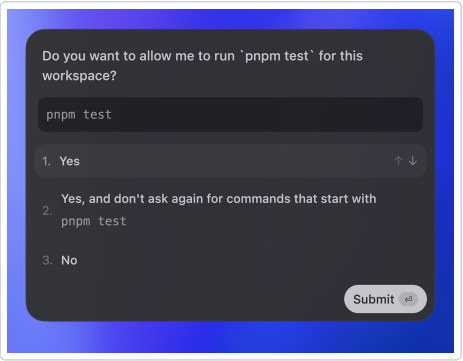
Same idea, different names: `AGENTS.md`, `CLAUDE.md`, `.cursor/rules`.

Tools, permissions, sandbox

- **read** browse and summarise
- **edit** change files in the workspace
- **run** shell commands and scripts
- **network** outside calls, uploads

each step down widens what a mistake can reach

Tool: a function the harness lets the model call.
Permission: the line between proposing and doing.
Sandbox: a place where doing is reversible.



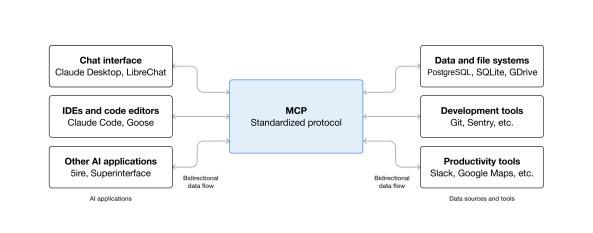
Codex asks before running a command. OpenAI, *Unlocking the Codex harness*, 2026.

MCP: one plug for many tools

Model Context Protocol

An open standard for connecting AI applications to external systems. The harness is the client; each server exposes tools, data and prompts.

Model Context Protocol (2026)



Cost to remember: every connected server adds its tool descriptions to the context before the task starts.

Source: diagram from modelcontextprotocol.io, read 26 Oct 2026.

Skills: procedures the agent loads on demand

```
---
name: regression-table
description: Turn stored Stata estimates into a paper-ready LaTeX
  table with esttab. Use when the user asks for a regression table,
  a results table, or to export estimates to LaTeX.
---

# Regression table

1. Check that `esttab` is installed: `which esttab`.
   If missing, stop and ask.
2. Store each model with `estimates store mN` right after
   the regression.
3. Export with:

   esttab m1 m2 using "output/table1.tex", replace ///
       se label booktabs star(* 0.10 ** 0.05 *** 0.01) ///
       stats(N r2, labels("Observations" "R-squared"))

4. Open the `.tex` file and check: one column per model,
   standard errors in parentheses, observations in the last rows.
5. Tell the user which file was written and how many columns
   it has.
```

A real skill from the demo repository.

> **Skill**
>
> A folder with a `SKILL.md` file: a name, a description, instructions, and optional scripts and references.
>
> Agent Skills (2026)

Loaded in three steps

Discovery: name and description only.
Activation: the full file, when a request matches.
Execution: scripts and references, if needed.

Nine of eleven production harnesses read `SKILL.md`; eight speak MCP (Barbaste et al., 2026).

Sub-agents and hooks

Sub-agent

An agent started by the main agent, with its own context and one narrow job. It hands back a short report and keeps its history.

A reviewer that reads the diff before a commit.

An explorer that searches a large repository.

A Stata runner that returns only the table.

Hook

A fixed command the harness runs at a set moment, whatever the model thinks. It is code; the model cannot skip it.

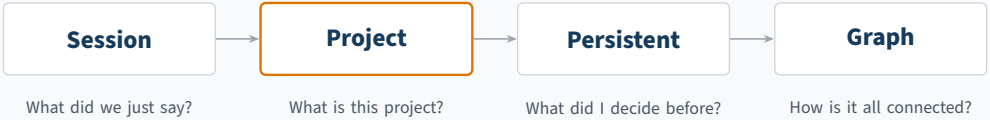
At session start: load the rules and the project state.

After an edit: run a linter or a prose check.

Before a commit: block secrets and raw data.

Sub-agents delegate judgement. Hooks remove it. Lesson 4 builds both.

Memory has layers



The context window is only the first layer. Compaction is why the others exist.

Lesson 2 builds the project layer: the one researchers use least.

Live: context, MCP and skills

What we do

Watch the context meter while the agent works, then run `/compact` and read the summary it keeps.

List the MCP servers the harness can reach.

Ask for “a regression table” and see which skill it loads.

What to look at

What the summary drops.

Which tools appear without being asked for.

When `SKILL.md` gets read.

Live on screen share. Demo repository: `https://github.com/MarcoLeMoglie/demo-lesson1`. If the demo fails, the next slide shows the recorded result.

Recorded result: the skill was read when the request matched

```
> build · deepseek-v4.1-flash-free
→ Read code/analysis.do
→ Read .agents/skills/regression-table/SKILL.md
[...]
← Edit code/analysis.do
-esttab m1 m2 using "output/table1.tex", replace se label booktabs
+esttab m1 m2 using "output/table1.tex", replace ///
+    se label booktabs star(* 0.10 ** 0.05 *** 0.01) ///
+    stats(N r2, labels("Observations" "R-squared"))
$ .../stata-mp -b do code/analysis.do; echo "EXIT:$?";
  tail -20 analysis.log
(output written to output/table1.tex)
→ Read output/table1.tex
Table written to `output/table1.tex` with 2 columns (m1, m2),
robust SEs in parentheses, observations (74/69), R-squared,
and `* p<0.10, ** p<0.05, *** p<0.01`.
```

Real run, 26 Oct 2026: OpenCode with DeepSeek V4.1 Flash (free tier), 57 seconds. Paths shortened.

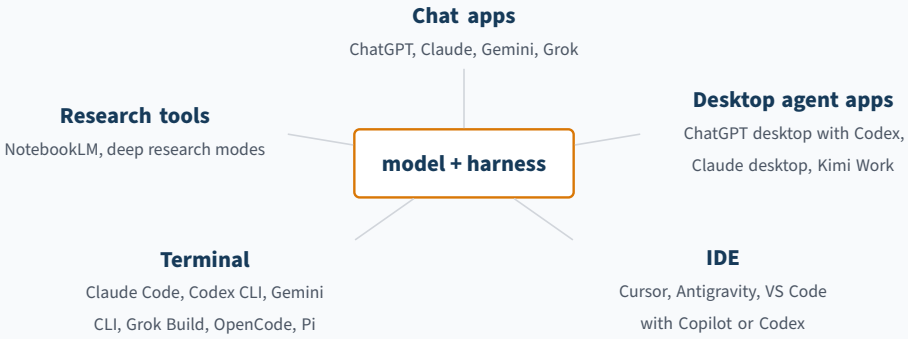
The request said “regression table”. The skill’s description said the same. The agent opened `SKILL.md` second, before touching Stata.

Stars, statistics and the closing report come from the skill. The prompt never asked for them.

Harnesses in 2026

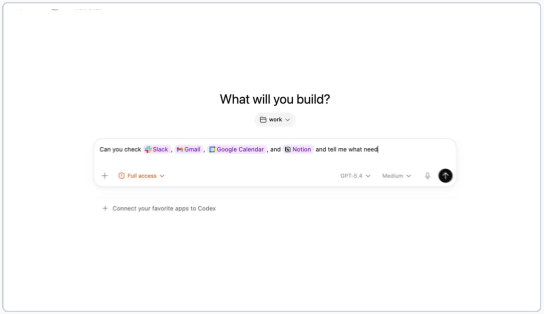
Where you meet agents, and why the window matters less

Five surfaces, one idea

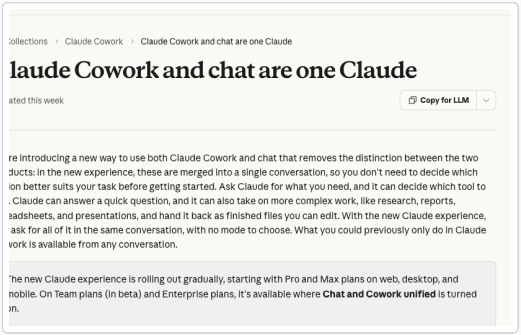


Pick the surface for the task. The loop underneath is the same.

Chat became agent



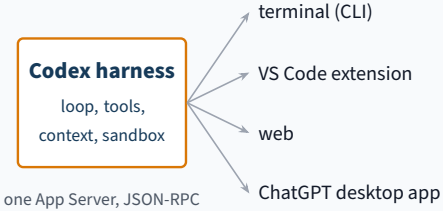
ChatGPT desktop app: Chat, Work and Codex in one window since 9 Jul 2026. OpenAI, learn.chatgpt.com.

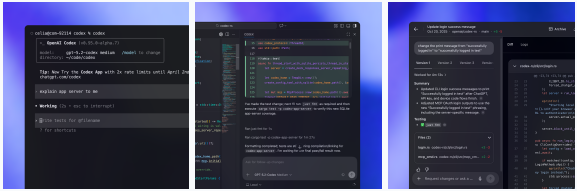


Claude: Chat and Cowork merged into one conversation, rolling out from 16 Sep 2026. Anthropic Help Center.

Both vendors removed the switch between chatting and delegating. The model now decides when to use tools, so you decide what it is allowed to touch.

One harness, many surfaces: Codex





Terminal, VS Code and web. Images from OpenAI, *Unlocking the Codex harness*, 2026.

Same agent, same rules file, same permissions in every window.

Choose the harness. The window is a detail.

Terminal and open harnesses

```
$ opencode models
[...]
opencode-go/claude-haiku-5-5
opencode-go/deepseek-v4.1-flash
opencode-go/gemini-4-argon
opencode-go/glm-5.3
opencode-go/gpt-6-luna
opencode-go/grok-4.7
opencode-go/kimi-k3.1-pro
opencode-go/mimo-v2.6-pro
opencode-go/minimax-m3.1-flash
opencode-go/qwen3.8-max
[...]   (39 models in total on this machine)
```

OpenCode on this laptop, 26 Oct 2026: one harness, models from ten labs.

Source: harness list from Barbaste et al. (2026).

Vendor harnesses

Claude Code, Codex CLI, Gemini CLI, Grok Build. Tuned for the vendor's own models.

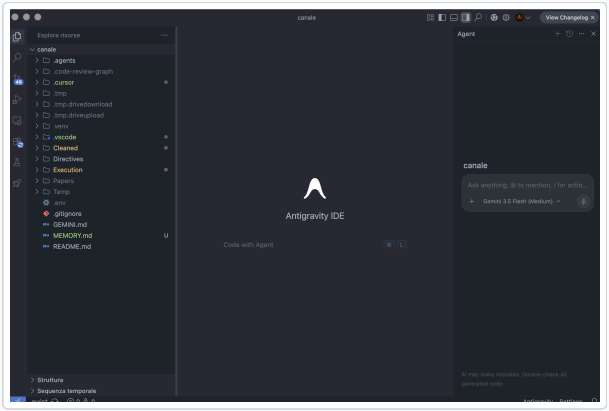
Open harnesses

OpenCode, Pi, Aider, OpenHands. Open source; you bring the model, through an API key or a router such as OpenRouter.

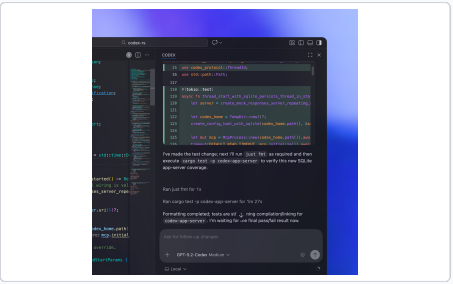
Why the terminal

Scriptable, runs on a server, and every step is text you can log.

IDE agents: review is a diff



Google Antigravity. Capture by the author, 2026.



Codex inside VS Code. OpenAI, 2026.

Every change arrives as a diff you accept or reject, line by line.

Start from the task

If the job is	**start with**
Thinking, writing, reading a paper	**a chat app**
Sources and evidence across many documents	**a research tool**
Work on a project folder, with files you review	**a desktop agent app or an IDE**
Batch jobs, servers, anything you want logged	**a terminal harness**
Data that must not leave the machine	**a local model in an open harness**

Most real work combines two of these.

Models and benchmarks

Who builds them, how to compare them, what people use

Who builds frontier models, October 2026

United States, closed weights

Anthropic	Claude Opus 5.5, Sonnet 5.5, Fable 5.1, Haiku 5.5
OpenAI	GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna
Google	Gemini 4 Argon
Meta	Muse Wonder, Muse Wonder Full
SpaceXAI	Grok 4.7

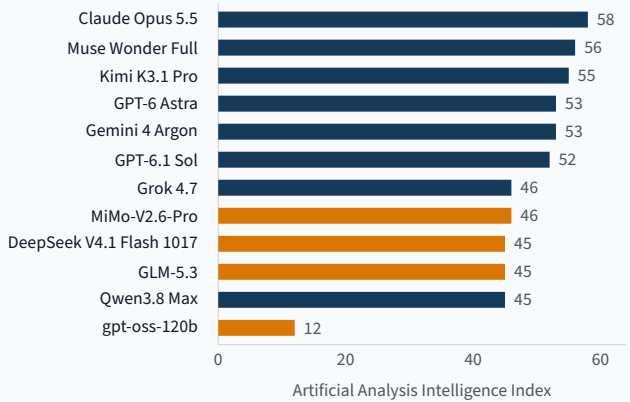
China, mostly open weights

DeepSeek	DeepSeek V4.1 Flash
Z.ai	GLM-5.3, GLM-5.3 Flash
Moonshot	Kimi K3, Kimi K3.1 Pro
Alibaba	Qwen3.8
Xiaomi	MiMo-V2.6 Pro and Flash

Frontier releases now arrive weeks apart: Grok 4.7 on 21 Sep, Opus 5.5 on 22 Sep, Muse Wonder on 12 Oct, Kimi K3.1 Pro on 23 Oct.

Source: vendor release pages; Artificial Analysis model list, read 26 Oct 2026.

One independent index



Orange: weights you can download today.
Blue: API only.

The best open-weight model sits 12 points below the top.

Kimi K3.1 Pro is called open-weight at launch; the weights are promised for 4 Nov. Until then, it is a closed model.

gpt-oss-120b is shown as the strongest US open-weight reference.

Source: Artificial Analysis (2026), best setting per model, read 26 Oct 2026; `aa_index_2026-10.csv`.

How to read a benchmark

Who ran it

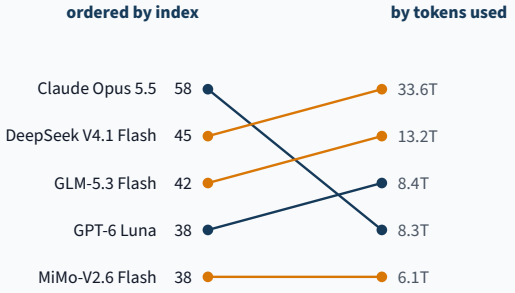
An independent lab, or the vendor's own launch post? Launch charts often reuse rivals' self-reported numbers.

At which setting

Claude Opus 5.5 scores 58 at maximum effort and 42 at low effort. Same model, same week.

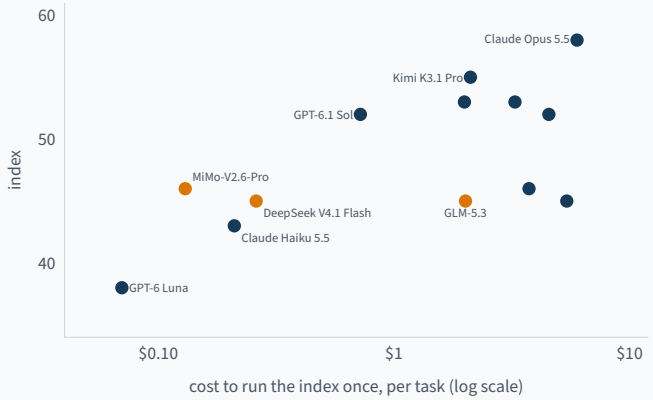
Ranked by what

Capability and use rank models differently. On the right, the same five models on two scales.



Index: Artificial Analysis. Tokens: OpenRouter, week to 25 Oct 2026.

Intelligence per dollar



Opus 5.5 costs $5.98 per task and scores 58.

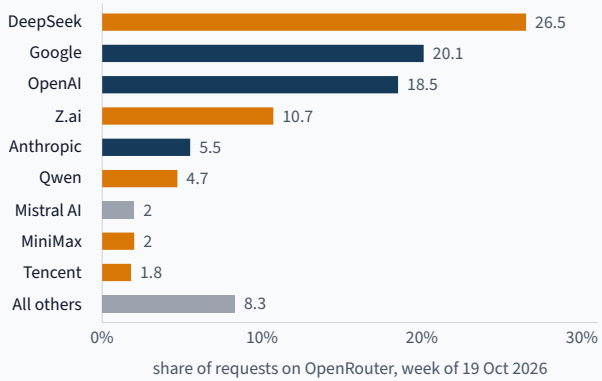
GPT-6.1 Sol costs $0.72 and scores 52.

MiMo-V2.6-Pro, open weights, costs $0.13 and scores 46.

About 45 times cheaper for twelve points. Whether that trade is good depends on the task.

Source: Artificial Analysis (2026), “cost per task” column, read 26 Oct 2026; `price_vs_index.csv`.

What people actually use



Orange: Chinese labs. **Blue**: US labs.

Open-weight models have carried more than 62% of OpenRouter tokens every week since July 2026.

This is traffic on one router, counted in requests. It is not market share, users or spend.

Source: OpenRouter (2026a) (CC BY 4.0); OpenRouter (2026b).

Open weights, open source, local

Open weights

The trained parameters can be downloaded and run on your own hardware, under the terms of a licence.

DeepSeek, GLM, Qwen, MiMo, Kimi K3, gpt-oss, Gemma.

Open source AI

Open weights, plus the training code and enough information about the data to rebuild the model.

Open Source Initiative, Open Source AI Definition 1.0. Few frontier models qualify.

Licences differ

Kimi K3's licence requires a separate agreement from model-as-a-service businesses above $20M in yearly revenue. Read the licence before building on a model.

Local pays off when

data cannot leave the machine, or the same small task runs thousands of times.

Local hurts when

the task needs frontier reasoning or long agent loops on a laptop. Tools: Ollama, LM Studio.

Live: same harness, another model

What we do

Run the bug-fixing task from the first demo in OpenCode.

Change one flag, `-m`, and run it again with a different model.

What to look at

Time to finish.

Number of steps.

Whether “done” is really done.

Live on screen share. Demo repository: `https://github.com/MarcoLeMoglie/demo-lesson1`. If the demo fails, the next slide shows the recorded result.

Recorded result: one task, three models

Model	Weights	Time	Tool calls	Result
DeepSeek V4.1 Flash (free tier)	open	22 s	9	fixed, table written
GLM-5.3	open	29 s	6	fixed, table written
GPT-6 Luna	closed	42 s	12	fixed, table written

Same harness, same prompt, same repository, same day. Only the `-m` flag changed. A one-line typo is easy for all three; harder tasks are where models separate.

Source: runs on 26 Oct 2026; full logs in `assets/lesson1/traces/`.

Cost, data, reliability

The minimum vocabulary before Lesson 3

Subscription vs API, and what drives the bill

Subscription

Flat monthly price. Limits are real but soft and change without notice. Fine for chat and desktop apps.

API

Pay per token. Easy to measure, easy to overspend. Needed when a harness runs many steps on its own.

List prices per million tokens, input and output: GPT-6 Luna $0.10 and $0.50; GPT-6.1 Sol $2 and $10; Claude Opus 5.5 $4 and $20; GPT-6 Astra $10 and $50.

Source: OpenAI and Anthropic API pricing pages, read 26 Oct 2026.

Four levers of an agent's bill

Context	every step resends files, history and tool output
Tool calls	more autonomy, more steps
Retries	failed runs, self-checks, sub-agents
Model tier	a 100-fold range between Luna and Astra

A chat ignores three of these. An agent pays for all four.

What a cost spike looks like

Workload	Example	Tokens, roughly
Chat question	“Explain heteroskedasticity in two lines.”	0.5k
Long reading	“Summarise this 30-page paper.”	25k
Agent task	“Rerun all tables and check them.”	100k to 200k
Long agent run	“Replicate Section 4 end to end.”	0.5M to 2M
Full context, ten retries	a 1M-token window resent ten times	10M

Ten resends of a 1M-token prompt on GPT-6.1 Sol: $40 at the long-context list price, about $2 if the context is cached.

Same window on screen. Four orders of magnitude in cost.

Does it train on my data, and what is risky

High risk

microdata, administrative or embargoed data, referee reports, IRB-restricted material

Medium

unpublished drafts, grant ideas, coauthors' notes, interview transcripts

Lower

public documentation, published papers, teaching material, synthetic data

Whether it trains depends on

the plan (consumer or business), the workspace settings, the API retention policy, and every tool the agent calls on the way.

Settings change. Check them; do not assume them.

When in doubt: anonymise, use a synthetic copy, or run locally.

Reliability words you will keep hearing

Hallucination	a confident output with no basis: a citation, a number, a variable
Context rot	quality falls as the context window fills
Reward hacking	the agent meets the check and misses the goal, for example by editing the test instead of the code
Sycophancy	the model agrees with you when you are wrong
Automation bias	we trust automated output more than it deserves (Skitka et al., 1999)
Drift	the same prompt gives different results after a model update

Lesson 3 is about controlling all six.

Twelve words, one line each

What an agent is

Model: the network that predicts text.

Harness: the runtime around it: loop, tools, context, controls.

Agent: model plus harness, given a task.

Tool and permission: what it can call, and what it may do without asking.

What it sees

Prompt: one request, one answer.

Context window: everything the model sees in one turn.

Token: the unit it reads, writes and bills.

Compaction: old turns replaced by a summary when the window fills.

How you extend it

Rules file: instructions loaded at every start, such as `AGENTS.md`.

MCP: an open standard that plugs tools and data into a harness.

Skill: a `SKILL.md` procedure loaded when a request matches.

Sub-agent: a helper agent with its own context and one job.

Before Lesson 2

Install one harness.

ChatGPT desktop with Codex, Claude Code, Cursor or OpenCode.

Connect one MCP server.

Any one: a file system, your calendar, Stata.

Write a ten-line `AGENTS.md` for a real project.

Layout, what must not change, what “done” means. Bring it to Lesson 2.

Demo repository: `https://github.com/MarcoLeMoglie/demo-lesson1`

References I

Agent Skills. Agent skills specification, 2026. URL https://agentskills.io/specification. Accessed 26 October 2026.

Anthropic. Effective context engineering for AI agents. Anthropic Engineering blog, 2025. URL https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Anthropic. Claude cowork and chat are one claude, September 2026. URL https://support.claude.com/en/articles/16761823-claude-cowork-and-chat-are-one-claude.

Artificial Analysis. LLM leaderboard: Intelligence index v4.3.2, 2026. URL https://artificialanalysis.ai/leaderboards/models. Accessed 26 October 2026.

Paul Barbaste, Tristan Darrigol, Germain Vu, and Tom Wiltberger. Harness engineering: Anatomy, architecture, and evolution of coding agents. a source-code study of eleven systems. arXiv:2609.00006, 2026. URL https://arxiv.org/abs/2609.00006.

Sanderson Oliveira de Macedo. What makes a harness a harness: Necessary and sufficient conditions for an agent harness. arXiv:2606.10106, 2026. URL https://arxiv.org/abs/2606.10106.

METR. Task-completion time horizons of frontier AI models, 2026. URL https://metr.org/time-horizons/. Time Horizon 1.1; page frozen 8 September 2026.

Model Context Protocol. What is the model context protocol (MCP)?, 2026. URL https://modelcontextprotocol.io/docs/getting-started/intro. Accessed 26 October 2026.

OpenAI. ChatGPT is now a partner for your most ambitious work, July 2026a. URL https://openai.com/index/chatgpt-for-your-most-ambitious-work/.

OpenAI. Unlocking the codex harness: How we built the app server, 2026b. URL https://openai.com/index/unlocking-the-codex-harness/.

References II

OpenRouter. LLM rankings: Market share by model author, 2026a. URL https://openrouter.ai/rankings. Week beginning 19 October 2026; data CC BY 4.0.

OpenRouter. The token state of play. Inferred, issue 01, September 2026b. URL https://openrouter.ai/data/newsletter/the-token-state-of-play.

Linda J. Skitka, Kathleen L. Mosier, and Mark Burdick. Does automation bias decision-making? International Journal of Human-Computer Studies 51(5): 991–1006, 1999.